

Reviewer Pack — Coxeter Group Action Complexity of Boolean Functions via Sym...

Entry b3bd26428f9d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 01:55:17 UTC

Coxeter Group Action Complexity of Boolean Functions via Symplectic Leaves

Entry ID: b3bd26428f9d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 01:55:17 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Complexity Theory: Boolean Function Entropy

Statement:

[‘For any boolean function f with n variables, there exists a representation $\rho(f)$ induced from the symplectic leaves of the moment polytope of f such that the action complexity of f is given by $\rho(f)$. Specifically, for all boolean functions f , $E[\rho(f)] = \Theta(2^{n/2})$ and the distribution of $\rho(f)$ follows a Gaussian with mean 0 and variance n .’, ‘For any two distinct boolean functions f_1 and f_2 , there exists an orthogonal projection in the symplectic leaf space that can separate them, ensuring that $E[\rho(f_1)^2 + \rho(f_2)^2] \geq (n^2/4)$.’, ‘The number of symplectic leaves that contain a given boolean function is bounded by 2^n , and the action complexity of f can be computed from the intersection of these leaves.’]

Rationale (proposer’s reasoning):

[‘Symplectic geometry provides a geometric structure on which boolean functions can be represented. The moment polytope of a boolean function can be viewed as a symplectic manifold, and the symplectic leaves within this manifold encode information about the function.’, ‘By studying the action complexity of boolean functions via symplectic leaves, we may uncover a new way to understand the computational complexity of these functions.’, ‘The Gaussian distribution and orthogonal projections in the statement suggest that symplectic geometry could provide insights into the entropy and separability properties of boolean functions.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07ec05de9ef96b75

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The action complexity of boolean function f is supported if $E[\rho(f)]$ falls within $\Theta(2^{(n/2)}) \pm 3$ standard deviations of a Gaussian distribution with mean 0 and variance n , AND for all distinct functions f_1 and f_2 , the inequality $E[\rho(f_1)^2 + \rho(f_2)^2] \geq (n^2/4)$ is satisfied by at least 95% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "Boolean function entropy" AND "action complexity" - "moment polytope" AND boolean function AND Gaussian distribution - "orthogonal projection" in symplectic leaf space AND boolean function separation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=195.9s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def moment_polytope(f):
        n = int(math.log2(len(f)))
        polytope = []
        for i in range(2**n):
            if f[i] == 1:
                polytope.append(i)
        return polytope

    def symplectic_leaves(polytope):
        leaves = []
        for i in range(len(polytope)):
            leaf = [polytope[i]]
            for j in range(i+1, len(polytope)):
                if (polytope[j] & polytope[i]) == 0:
                    leaf.append(polytope[j])
```

```

        leaves.append(leaf)
    return leaves

def action_complexity(leaves):
    return sum(len(leaf) for leaf in leaves)

n = random.randint(1, 40)
f = generate_boolean_function(n)
polytope = moment_polytope(f)
leaves = symplectic_leaves(polytope)
rho_f = action_complexity(leaves)

mean = 2**(n/2)
std_dev = math.sqrt(n)
if not (mean - 3*std_dev <= rho_f <= mean + 3*std_dev):
    return {
        "metric_name": "Action Complexity",
        "metric_value": rho_f,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Action complexity does not follow the Gaussian distribution."
    }

return {
    "metric_name": "Action Complexity",
    "metric_value": rho_f,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((result["seed"] for result in results if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample='Action complexity does not follow the Gaussian distribution'")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test to ensure it completes successfully and produces the necessary data for evaluation.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13247
2	preregistration	ollama_remote	glm4:latest	0	0	7129
3	novelty	ollama_remote	glm4:latest	0	0	4962
4	novelty	ollama_remote	glm4:latest	0	0	5290
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17406
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11927
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25998
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10322
9	judge	ollama_remote	glm4:latest	0	0	57927

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154210 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b3bd26428f9d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b3bd26428f9d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b3bd26428f9d.tar.gz` (if generated)